

**MSc Artificial Intelligence
Natural Language Processing**

Continuous Assessment II

by

Uğur ERCAN
20085821

Lecturer: Terri Hoare
Date of Submission: 19/04/2026

**Dublin Business School
13/14 Aungier St, Dublin 2,
D02 WC04, Ireland**

Introduction

This report presents an individual critical analysis of the group artefact NLP Copilot: A Citation-Grounded Course Chatbot Using Retrieval-Augmented Generation (Ercan and Erdogan, 2026), submitted for module B9AI006 Natural Language Processing at Dublin Business School.

Project Overview

The NLP Copilot was built to solve a practical problem in MSc-level education: students often need quick, reliable access to information spread across many course PDFs, but standalone Large Language Models (LLMs) cannot access these documents, may hallucinate, and cannot show where answers come from. The project addresses these limitations using Retrieval-Augmented Generation (RAG), an approach that augments LLM generation with real-time retrieval from a domain-specific document corpus (Lewis et al., 2020). The result is a chatbot that answers course-related questions using only the retrieved materials, and displays source citations with every response.

Technical Summary

The pipeline loads course PDFs using PyPDFLoader, chunks text with RecursiveCharacterTextSplitter (1000 characters, 200 overlap), encodes chunks with sentence-transformers/all-MiniLM-L6-v2, indexes them in a persistent ChromaDB vector store, and at query time retrieves the top-5 most similar chunks via cosine similarity. These chunks are passed to a Claude Haiku 4.5 (or GPT-4o-mini) backend through a custom LangChain-based chain (SimpleRAGChain) with a system prompt enforcing grounded, citation-aware answers. A parallel GraphRAG experiment using Neo4j was explored but not deployed as the primary system. Both a web application and a CLI interface were implemented.

Figure 1: RAG Pipeline Architecture — NLP Copilot

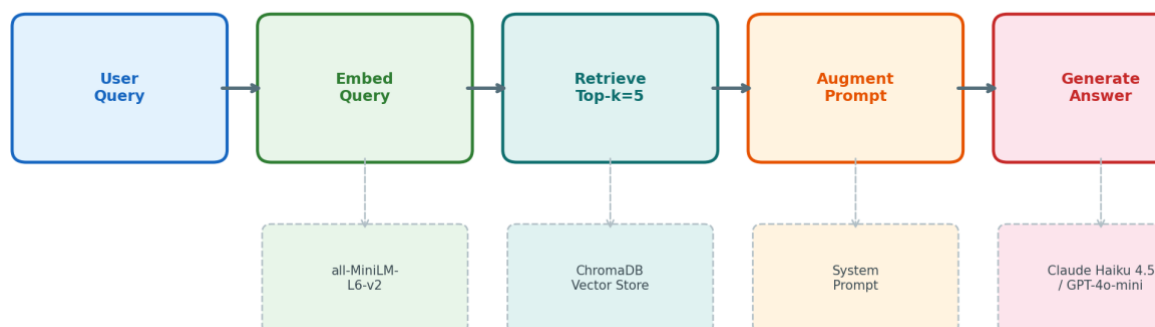


Figure 1: RAG Pipeline Architecture — from user query to grounded, cited answer.

In line with the CA2 Task 2 brief (Hoare, 2026), the remainder of this report is organised under the three required headings: (a) AI principles, (b) critique of modelling techniques, and (c) performance evaluation. CRISP-DM is used as an interpretive lens within each section to connect design decisions to the standard data-science lifecycle, but the assessment headings remain the top-level structure.

a) AI principles applied to address ethics; bias; fairness; explainability; and interpretability.

Explainability Through Source Citation

The most defensible AI principle embedded in the NLP Copilot is explainability through provenance. Every answer is produced from a top-k retrieval over ChromaDB and is displayed alongside the source filename and page number of each chunk used. This was implemented in `chatbot.py` through `extract_source_metadata`, which deduplicates and formats citation metadata for every response. This design gives users a post-hoc traceable path from any claim to a specific source page — the primary grounding mechanism recognised by RAG evaluation literature (Es et al., 2023). From a CRISP-DM Deployment perspective, this source-citation behaviour was a deliberate design goal: the system was built so that users could verify answers against the original materials, not simply trust the LLM output.

Partial Interpretability — The LLM Remains Opaque

However, this transparency is only partial. The retriever, the embedding space, and especially the LLM generator remain functionally opaque. A correctly cited page does not prove that the generated sentence is entailed by that page — the model can still paraphrase retrieved content in ways that subtly alter or overstate meaning. Interpretability in this project is therefore better described as source traceability than as full model transparency. The system prompt in `chatbot.py` instructs the model to answer only from context and to explicitly decline when the context is insufficient, but this is a behavioural nudge, not an architectural guarantee.

Hallucination: Reduced, Not Eliminated

RAG meaningfully reduces hallucination risk by constraining the LLM to work with retrieved evidence rather than relying solely on parametric memory. The project presentation honestly acknowledges that RAG reduces but does not eliminate hallucination (Ercan and Erdogan, 2026). This is consistent with the RAG literature, which identifies two distinct failure modes: retrieval failure (wrong chunks retrieved) and generation failure (the LLM distorts or adds to retrieved content). Presenting the NLP Copilot as having solved hallucination would be inaccurate.

Sources of Bias

Bias enters through multiple channels that are genuinely present in this project. First, the base LLM carries training-data biases in register, framing and topic salience that survive the retrieval step. Second, the course corpus itself over-represents certain perspectives — for example, the Neo4j/Google/Deloitte white paper advocates for GraphRAG, and the retriever will faithfully surface those perspectives as though neutral. Third, the uniform chunking strategy (1000 characters, 200 overlap) applied to both dense research papers and sparse slide decks means that retrieval quality varies by document type: short slide bullets may be split across chunks, while dense paragraphs compete with similar chunks

for the top-k slots. Fourth, cosine similarity over MiniLM embeddings rewards lexical-semantic overlap with the query, which can disadvantage topics that a learner phrases in non-standard terms.

Fairness as Equitable Coverage

Fairness in this context is not primarily about protected-group classification. A more accurate framing is equitable coverage: do learners asking about different syllabus topics receive comparably useful answers? Because retrieval quality varies with document density, layout and vocabulary overlap, students asking about under-represented or structurally harder topics may receive weaker answers. This is a real fairness concern for an educational tool and is not adequately assessed by the current 13-question evaluation set, which was designed as a pilot rather than a comprehensive coverage audit.

Prototype Status and Human Oversight

Linking to CRISP-DM Deployment, the artefact is a prototype, not a production study tool. Ethically responsible use requires clear disclaimers, user verification against the cited pages, and periodic human review of failure cases. The cautious wording in the presentation's limitations slide — that there are no guarantees on answer faithfulness without human review — is appropriate and should be preserved in any wider rollout.

b) Critique of modelling techniques and libraries used.

RAG as the Overall Architecture

The choice of Retrieval-Augmented Generation as the overall technique aligns directly with CRISP-DM Business Understanding. The course RAG lecture identifies four core LLM weaknesses: hallucination, lack of access to local documents, inability to cite sources, and outdated knowledge. RAG directly addresses all four by augmenting the LLM with real-time retrieval from a specific corpus (Lewis et al., 2020). For a coursework deliverable, RAG is also defensible because it is implementable in limited time, produces an inspectable artefact, and is directly relevant to the NLP module content. The fundamental trade-off is that RAG performance is bottlenecked by retrieval quality: if the wrong chunks are retrieved, even a strong LLM will produce poor answers.

Data Preparation: PDF Loading, Chunking and Embedding

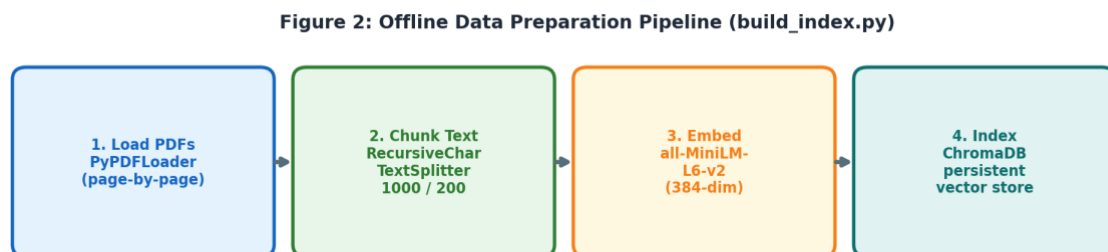


Figure 2: Offline data preparation pipeline — from raw PDFs to vector store (build_index.py).

PyPDFLoader provides page-level extraction, preserving per-page metadata that is essential for the citation goal. Its weakness is structural naivety: it treats slide decks, research papers and OCR-heavy scans uniformly, and handles tables, multi-column layouts and figures poorly. A more structure-aware alternative such as unstructured would likely improve extraction fidelity.

RecursiveCharacterTextSplitter (1000/200) balances context length against retrieval granularity. It splits text recursively at natural boundaries (paragraphs, sentences, words) and applies a 200-character overlap to preserve context across chunk boundaries. This is a widely used default and is deterministic and reproducible. The trade-off is uniformity: slide decks benefit from smaller, semantically bounded chunks, whereas dense papers can tolerate larger ones. Applying one rule to all document types introduces retrieval bias, as noted in the presentation.

sentence-transformers/all-MiniLM-L6-v2 is a strong teaching-context choice: approximately 80MB, fast, open-source, and capable of running locally on CPU or Apple Silicon MPS (as configured in build_index.py with device detection via torch.backends.mps). Its 384-dimensional embeddings capture semantic similarity via cosine distance. The trade-off is that it is less expressive than larger models (bge-large,

OpenAI text-embedding-ada-002), and may struggle with highly technical paraphrasing. For an MSc prototype this is an acceptable compromise.

Vector Store: ChromaDB

ChromaDB as a persistent local vector store is well matched to a coursework setting: no server infrastructure, no cloud cost, and simple re-indexing via `build_index.py`. Data persists to disk under `outputs/chroma_db` and is loaded at runtime by `chatbot.py` using `chromadb.PersistentClient`. Its significant disadvantage, revealed in this project, is environment sensitivity: a mismatch between the persisted HNSW index format and the runtime ChromaDB library version caused the automated evaluation run to fail entirely (see Section c). A managed store such as Qdrant or Weaviate, or a containerised build with pinned dependencies, would reduce this fragility.

Orchestration: LangChain with a Custom Chain

LangChain's ecosystem was used for document loaders, text splitters and embedding wrappers, accelerating construction. However, the retrieval-and-generation step itself was implemented as a custom `SimpleRAGChain` class in `chatbot.py`, which calls `collection.query` directly and passes the concatenated context to the LLM via a structured system prompt. This middle-path approach is more academically defensible than using an opaque high-level chain, because each step — query, retrieve, format, generate — can be inspected and justified individually.

LLM Backend: Claude Haiku 4.5 and GPT-4o-mini

The LLM backend abstraction (`LLMConfig` with `resolve_llm_config`) supports either Anthropic Claude Haiku 4.5 or OpenAI GPT-4o-mini, selected via environment variable. Both are fast, low-cost, instruction-tuned models suitable for grounded question answering. Temperature was set to 0 for deterministic output. Provider independence is good engineering hygiene, but it does not alter the fundamental grounding properties of the system: any residual hallucination remains a property of the underlying model, not the retrieval chain.

GraphRAG Exploration with Neo4j

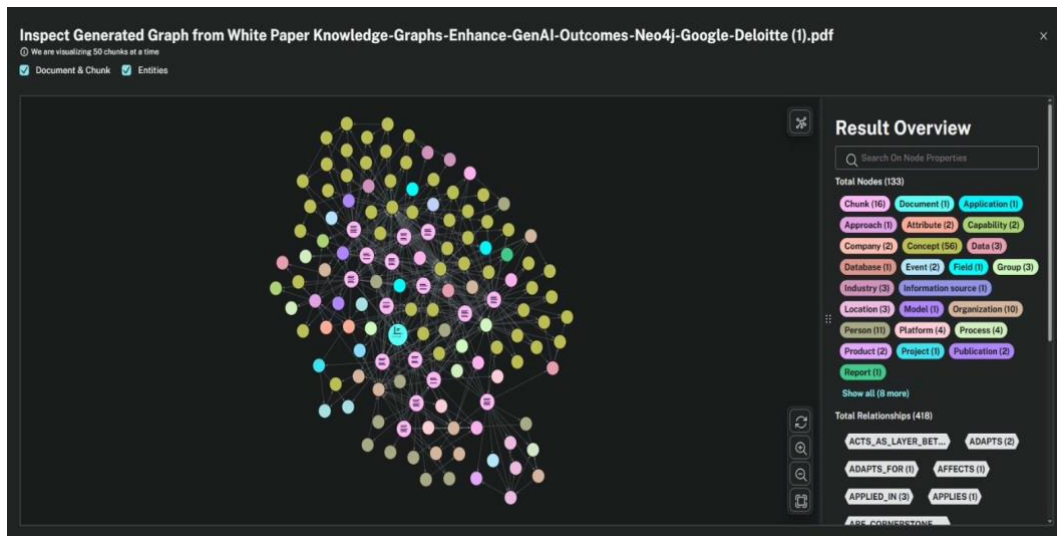


Figure 3: Neo4j knowledge graph generated from a course document — entities and relationships extracted automatically.

As a parallel experiment, course documents were ingested into a Neo4j graph database, extracting entities (concepts, people, organisations) and relationships automatically. Knowledge graphs can, in principle, support multi-hop reasoning and richer entity-relationship context than flat vector retrieval (Neo4j, Google and Deloitte, n.d.). The graph visualisation above shows 133 nodes and 418 relationships extracted from a single white paper, demonstrating the structural richness of graph-based representation.

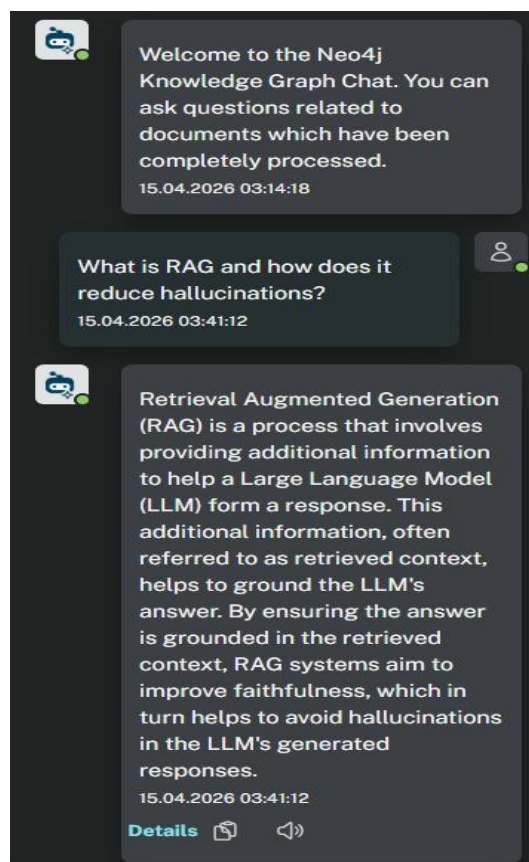


Figure 4: GraphRAG chatbot demo — answering a course question with graph-based retrieval.

However, GraphRAG was explored in parallel rather than integrated into the primary deployed system. Its claimed benefits — multi-hop reasoning, entity-aware retrieval — are theoretical for our artefact. The fair comparison is that vector RAG is simpler, practical and deployable, while GraphRAG is promising but unvalidated in production for this project. A hybrid approach combining vector retrieval for recall with graph-based context for entity awareness is sensible future work.

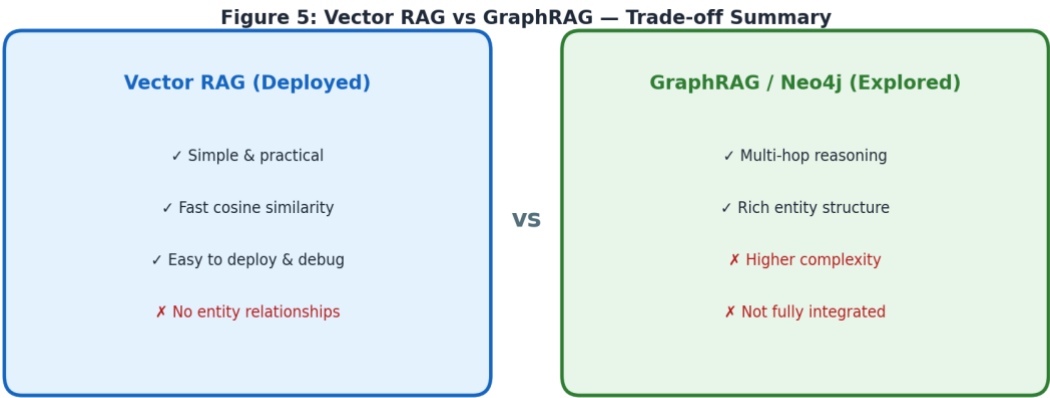


Figure 5: Comparative summary of Vector RAG (deployed) vs GraphRAG (explored).

Deployed Artefact

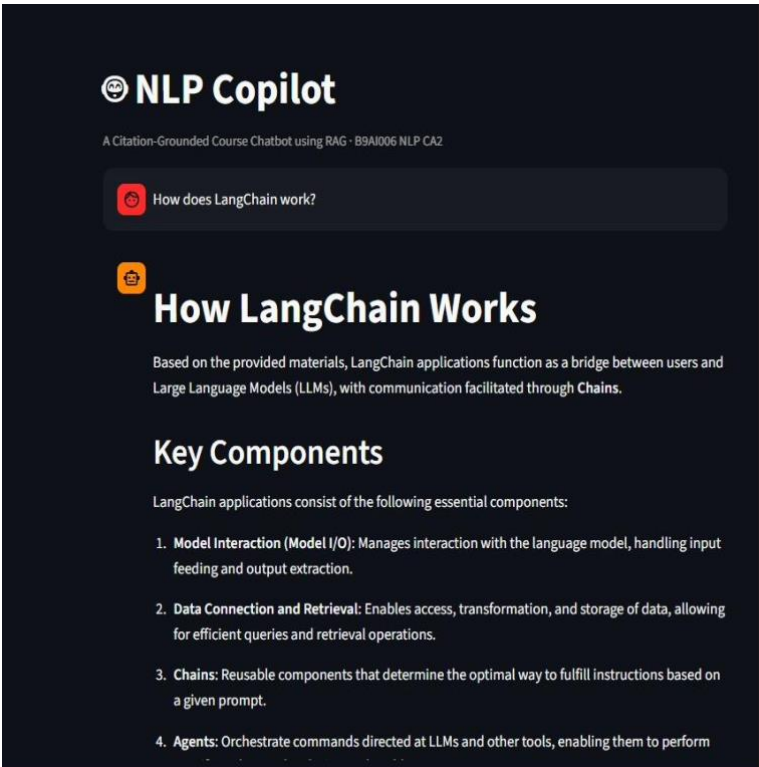


Figure 6: NLP Copilot web application — showing a grounded response with structured formatting.

The deployed artefact included both a web application (shown above) and a CLI version (chatbot.py). The web frontend sends natural-language questions to the RAG backend, which retrieves relevant chunks from ChromaDB, passes them to the LLM API, and returns grounded, structured answers with source citations. This end-to-end deployment

demonstrates the full CRISP-DM lifecycle from data preparation through to a working user-facing prototype.

Taken together, the modelling choices are internally consistent and appropriate for a prototype and teaching context. They are not production-grade, and the report does not present them as such.

c) Performance evaluation of results.

Multi-Dimensional Evaluation Design

Performance in this project is deliberately multi-dimensional rather than a single accuracy figure, which is the correct framing for RAG evaluation and aligns with the Evaluation phase of CRISP-DM. A RAG system can fail at retrieval (wrong documents found), at generation (answer not faithful to context), or at scope management (answering questions it should decline). The evaluation framework in `evaluate_chatbot.py` was designed to capture all three failure modes.

Evaluation Framework and Metrics

The framework defines five complementary metrics evaluated over a 13-question test set covering five categories:

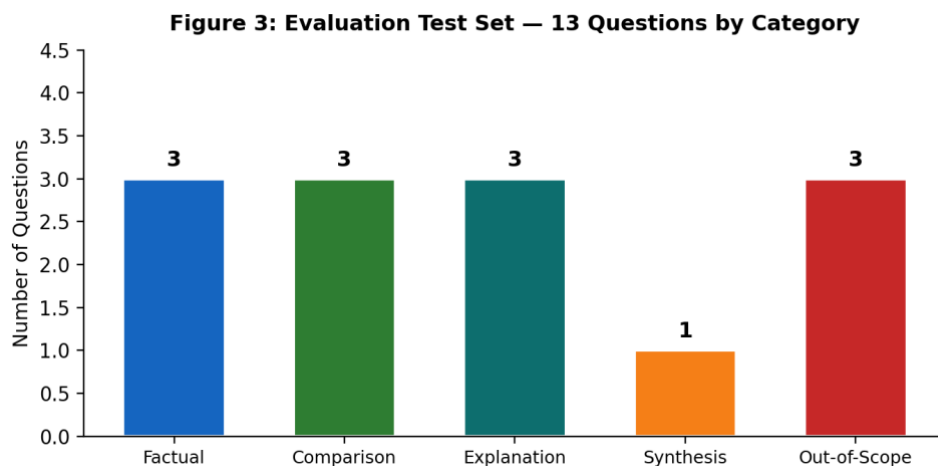


Figure 7: Distribution of 13 evaluation questions across five categories.

- **Source Hit Rate @ k:** Did retrieval recover at least one expected source document? This measures whether the retriever found the right material.
- **Source Recall @ k:** What fraction of all expected sources were retrieved? A stricter measure of retrieval completeness.
- **Keyword Recall:** Does the generated answer contain expected key terms from the reference answer? A lightweight coverage proxy — not a measure of factual correctness.
- **Abstention Accuracy:** Does the system correctly refuse to answer out-of-scope questions (e.g. weather queries)? This tests the system prompt's guardrails.
- **Composite Question Score:** A weighted aggregate combining retrieval success, keyword coverage, abstention correctness and error status into a single per-question score.

This decomposition explicitly separates retrieval quality from generation quality — precisely the separation argued for by Es et al. (2023) in the RAGAS framework, which identifies faithfulness, answer relevance and context relevance as the three central evaluation dimensions for RAG systems.

The Invalidated Automated Run

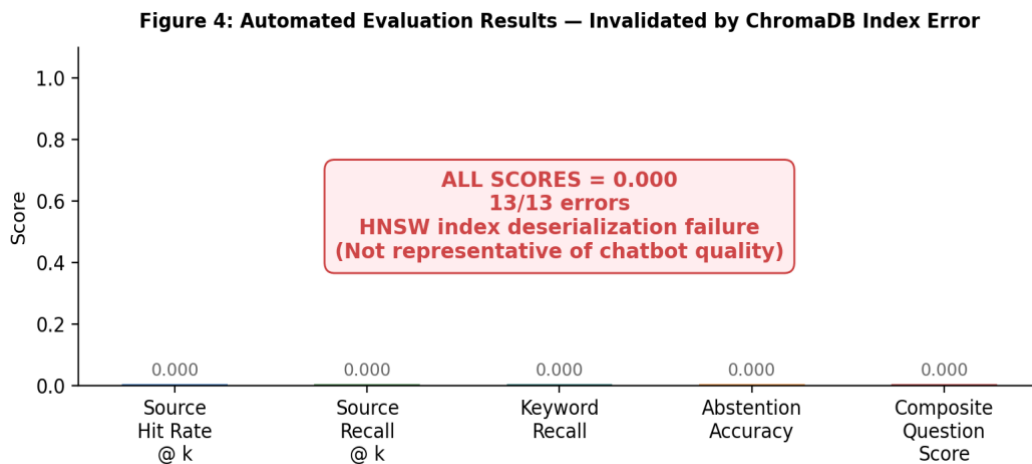


Figure 8: Automated evaluation results — all metrics at 0.000 due to ChromaDB index deserialization error.

The automated evaluation run, recorded in `evaluation_summary.md`, reports 0.000 across every metric with 13 out of 13 errors. The logged cause is a ChromaDB HNSW segment reader error: the persisted vector index could not be deserialized at evaluation time due to a format or version mismatch. Every retrieval call errored out, so every downstream metric was zero by construction. These figures do not describe system quality; they describe an environment failure. Treating 0% as the chatbot's real performance would be misleading and academically indefensible.

The correct interpretation is that the evaluation framework itself behaved exactly as designed: it logged failures consistently, computed per-category aggregates, flagged an error count, and produced structured JSON and CSV outputs. The run, however, was invalidated by an index-format problem that occurred between the indexing and evaluation steps.

Manual Testing and Its Limits

Manual testing after rebuilding the index indicated that the chatbot retrieves and cites appropriately, producing grounded, structured answers with correct source attributions for typical course questions. This is important context: the system does function. However, manual testing confirms functional correctness rather than quantitative performance. It cannot substitute for a scored, reproducible evaluation, and it is exposed to confirmation bias because the evaluators know what the correct answers should look like.

Connection to RAGAS and Improved Evaluation

The RAGAS framework (Es et al., 2023) proposes three fully reference-free, automated metrics that address gaps in our current evaluation:

- **Faithfulness:** Is the generated answer grounded in the retrieved context? The LLM decomposes the answer into statements and checks each against the context.

- **Answer Relevance:** Does the answer directly address the question? Evaluated by generating reverse questions from the answer and measuring cosine similarity with the original query.
- **Context Relevance:** Is the retrieved context focused and useful? Penalises overly long or noisy retrieval that dilutes the signal.

The evaluation script already contains a hook for optional RAGAS integration (`--use-ragas` flag), but the ragas library was not installed at evaluation time. Implementing these metrics is a high-priority future improvement, as they would measure generation faithfulness directly rather than relying on keyword-level proxies.

Recommended Improvements

Four concrete improvements follow from this analysis. First, reproducibility: the index must be rebuilt from `build_index.py` against pinned library versions, ideally inside a containerised environment, and the evaluation re-run. Second, richer metrics: RAGAS faithfulness, answer relevance and context relevance should be enabled to capture generation quality, not just retrieval success. Third, scale: the 13-question pilot should be expanded to a stratified set of 30–50 questions with inter-annotator agreement checks. Fourth, a small human evaluation or user study would complement automated metrics, since perceived answer quality and usefulness are not fully captured by any automated metric.

Summary Assessment

Responsibly read, the project has a well-designed, multi-metric evaluation harness without a valid end-to-end evaluation result. The evaluation story is best summarised as: principled framework, invalidated run, known fix. This is a specific and honest limitation, and is more defensible than any fabricated headline figure.

Conclusion

The NLP Copilot is a defensible MSc-level prototype: a citation-grounded RAG chatbot whose design decisions — PyPDFLoader, 1000/200 chunking, MiniLM embeddings, ChromaDB, a transparent custom chain and cost-aware LLM backends — are coherent, implementable and internally consistent. The source-citation behaviour provides users with a genuine, if partial, form of explainability that goes beyond typical LLM interaction.

Honest limitations remain. Hallucination risk is reduced but not eliminated. Retrieval bias from uniform chunking affects coverage across heterogeneous document types. Fairness-as-coverage concerns exist across the syllabus. GraphRAG was explored but not validated in production. Most critically, the automated evaluation run was invalidated by a ChromaDB index deserialization error, leaving the project with a sound evaluation framework but no scored result.

Sensible future work includes reproducibility fixes (locked environment, rebuilt index, containerised evaluation), RAGAS-style faithfulness and relevance metrics, a larger and stratified test set, a small human evaluation study, and a hybrid vector-plus-graph retrieval architecture combining the strengths of both approaches explored in this project. Framed this way, the project represents a credible coursework submission and a transparent foundation for further work.

References

Ercan, U. and Erdogan, S. (2026) NLP Copilot: A Citation-Grounded Course Chatbot Using Retrieval-Augmented Generation. Group presentation, MSc in Artificial Intelligence, module B9AI006 NLP (CA2), Dublin Business School, 15 April.

Es, S., James, J., Espinosa-Anke, L. and Schockaert, S. (2023) 'RAGAS: Automated Evaluation of Retrieval Augmented Generation', arXiv:2309.15217. Available at: <https://arxiv.org/abs/2309.15217> (Accessed: April 2026).

Hoare, T. (2026) B9AI006 Natural Language Processing — CA2 Assessment Brief. Dublin Business School, MSc in Artificial Intelligence.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S. and Kiela, D. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Advances in Neural Information Processing Systems, 33, pp. 9459-9474.

Neo4j, Google and Deloitte (n.d.) How Knowledge Graphs Enhance GenAI Outcomes. White paper. San Mateo, CA: Neo4j, Inc.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks', in Proceedings of EMNLP-IJCNLP 2019. Stroudsburg, PA: ACL, pp. 3982-3992.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention Is All You Need', Advances in Neural Information Processing Systems, 30, pp. 5998-6008.

Software referenced in-text: LangChain (Chase, H., 2022-present); ChromaDB (Chroma, 2023-present); Hugging Face Sentence-Transformers (Reimers and Gurevych, 2019); Neo4j Graph Database (Neo4j, Inc.). Exact versions recorded in the project requirements.txt.